
Generalizing Well

SAM, ASAM, GSAM & Lookahead

Lesson 8 of the Optimizer Course

Why the lowest point isn't always the best point —
and the optimizers that chase *flat* valleys

Every optimizer so far raced to the lowest training loss. But a model is judged on data it has never seen — and it turns out two minima at the same height can behave completely differently on new data. The secret is flatness. This lesson explains why flat beats sharp, and introduces optimizers that deliberately seek the flat valleys.

1 The twist: low isn't the same as good

Here's the uncomfortable truth about everything we've done. We've been minimising the *training* loss — the error on data the model has already seen. But what we actually care about is the *test* loss — the error on *new* data. They are not the same surface. The test surface is like the training surface nudged sideways a little (because the test data is slightly different from the training data).

Now picture two minima at the exact same training-loss height: one a **narrow, steep** ravine, the other a **wide, flat** basin. Nudge the surface sideways (train $\rightarrow$ test) and watch what happens to the loss at the bottom you found:

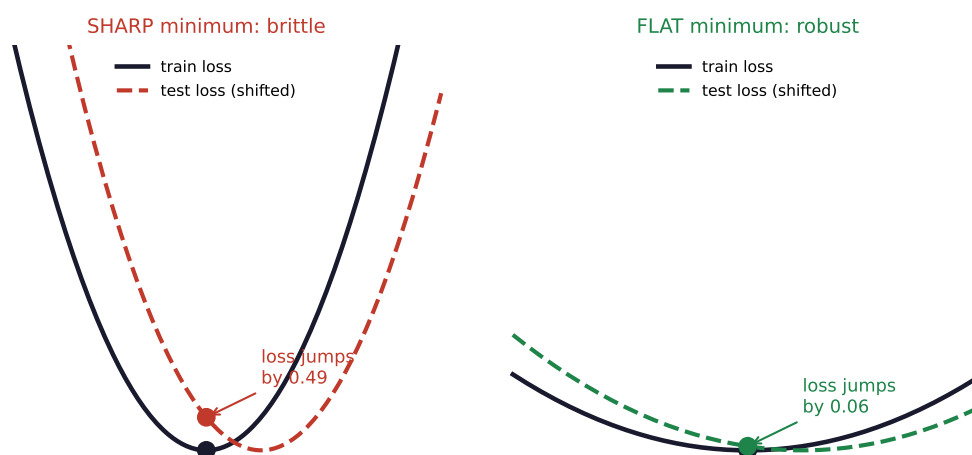
Let's actually do the numbers

A sideways shift of $\delta = 0.1$. The loss change at the old minimum is about $\frac{1}{2} \times (\text{curvature}) \times \delta^2$.

- **Sharp minimum** (curvature = 100): loss jumps by $\frac{1}{2} \cdot 100 \cdot 0.1^2 = \mathbf{0.50}$.

- **Flat minimum** (curvature = 1): loss jumps by $\frac{1}{2} \cdot 1 \cdot 0.1^2 = \mathbf{0.005}$.

The sharp minimum is **100 $\times$ more brittle** to the same shift. The flat one barely notices. That's the whole reason flatness $\approx$ generalisation.



The big idea

A model in a **flat** minimum is robust: small differences between training and real-world data barely change its loss, so it generalises. A model in a **sharp** minimum is brittle: the same small differences send its loss (and error) soaring. So we should hunt for minima that are not just *low* but *flat*.

This also explains a mystery from Lesson 5: why plain SGD often generalises *better* than Adam. SGD's gradient noise (Lesson 2) jitters it out of narrow, sharp ravines — they're unstable under noise — and lets it settle only in wide, flat basins that the noise can't shake it out of. Adam's smoother, adaptive steps can march confidently into a sharp minimum that SGD would have bounced out of.

2 SAM: minimise the worst loss nearby

Sharpness-Aware Minimisation makes the flatness goal explicit. Instead of asking “what’s the loss *here*?”, SAM asks “what’s the *worst* loss *anywhere in a small neighbourhood* around here?” — and minimises that. If the worst nearby loss is low, you must be in a flat region (a sharp minimum has a terrible worst-case point just up the wall).

It does this in two steps per update:

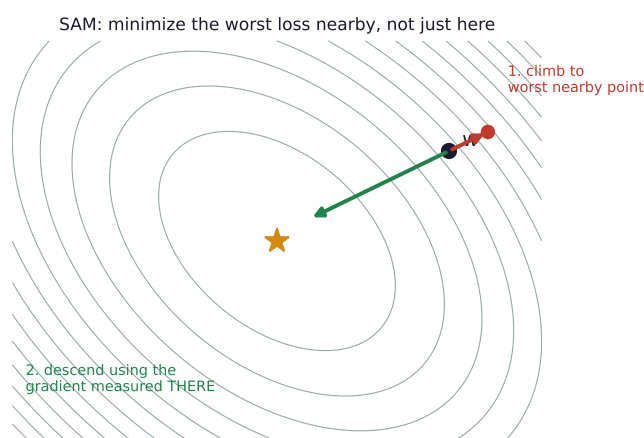
1. **Ascent (find the worst nearby point):** climb a small distance ρ in the gradient direction. The perturbation is $\hat{\varepsilon} = \rho \frac{\nabla f}{\|\nabla f\|}$.
2. **Descent (fix the flatness there):** compute the gradient *at that worse point*, and use *it* to update the original weights.

Let’s actually do the numbers

At $w = (2, 1)$ with gradient $g = (8, 3)$ and radius $\rho = 0.05$: $\|g\| = \sqrt{73} = 8.544$, so

$$\hat{\varepsilon} = 0.05 \cdot \frac{(8, 3)}{8.544} = (0.0468, 0.0176), \quad \text{worst nearby point} = (2.047, 1.018).$$

SAM then measures the gradient *there* and steps the real weights with it. The effect: it avoids directions where the loss spikes just next door — i.e. it steers toward flatness.



Watch out (common confusion)

SAM costs about **2**× a normal step — two forward-backward passes (one for the ascent, one for the descent). That’s a real price. You pay it when generalisation matters more than training speed; SAM has produced state-of-the-art results in vision precisely because of this.

Two refinements you’ll see named:

- **ASAM (Adaptive SAM).** Plain SAM’s fixed-radius ball is sensitive to how the weights happen to be scaled — rescale a layer and the “sharpness” it sees changes, even though the model is identical. ASAM makes the radius adapt per-weight, so its flatness measure is scale-invariant and more reliable.

- **GSAM (Surrogate-Gap SAM).** Minimises the *gap* between the perturbed loss and the clean loss — a direct measure of sharpness — alongside the loss itself, squeezing out flatter solutions still.

3 Lookahead: k steps forward, one step back

Lookahead is different in kind: it’s a *wrapper* you put around any optimizer (SGD, Adam, anything). It keeps two copies of the weights:

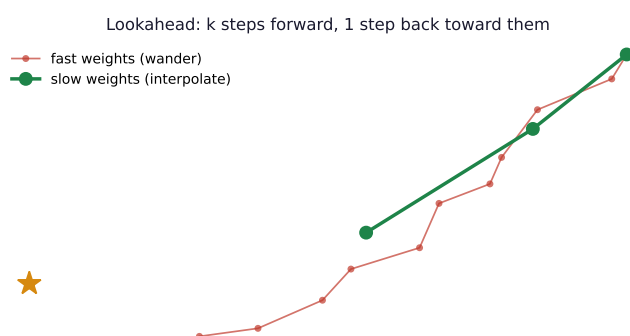
- **Fast weights** take k ordinary steps (say $k = 5$) with the base optimizer — wandering, possibly oscillating, exploring.
- **Slow weights** then jump a fraction α of the way toward wherever the fast weights ended up: $\text{slow} \leftarrow \text{slow} + \alpha(\text{fast} - \text{slow})$.

Let’s actually do the numbers

Slow weights at $(1, 1)$; after 5 fast steps the fast weights are at $(0.3, 0.7)$; with $\alpha = 0.5$:

$$\text{new slow} = (1, 1) + 0.5((0.3, 0.7) - (1, 1)) = (0.65, 0.85).$$

The slow weights move smoothly toward the fast ones’ destination, ignoring the wobble along the way.



This “explore with the fast weights, commit cautiously with the slow ones” dance dampens oscillation and reduces variance, often improving stability and final accuracy for almost no extra cost (just one extra copy of the weights). It pairs nicely with Adam (the combination was dubbed “Ranger”).

4 What you can do now, and what’s next

You can now explain why flat minima generalise better than sharp ones (with the brittleness calculation), why SGD’s noise biases it toward flatness, how SAM’s two-step ascent/descent targets flat regions, and what Lookahead’s slow/fast weights do.

Next — Lesson 9: Schedules & No-Tuning. We’ve treated the learning rate η as a single fixed number this whole time. In reality it’s almost always *changed over the course of training* — small at first, then large, then small again. Lesson 9 is about that hidden dimension: warmup, cosine decay, and the newest “you don’t even have to tune the learning rate” methods.